

CS 21 25.2 Project (Arch-252 Ecosystem)

Project Overview

The CS 21 25.2 project involves the creation of the following for a custom architecture called *Arch-252*:

1. RISC-V-based assembler
2. Pyxel-based emulator
3. Simplified Flappy Bird in Arch-252 assembly
4. Digital-based Arch-252 processor

You are to form a group with 3-4 students from CS 21 25.2. Each student must be the *main contributor* of at least one of the deliverables above.

You may think of the project as a solo activity in which you choose one of four given tasks (*in which the task must be unique within a group*) with collaboration between group members allowed.

Arch-252 Architecture

Overview

Arch-252 is a 16-bit architecture with the following architectural registers:

- `pc` (program counter)
- `sp` (stack pointer)
- `tm` (temporary register)
- `ga` (general-purpose register A)
- `gb` (general-purpose register B)
- `gc` (general-purpose register C)
- `gd` (general-purpose register D)
- `ge` (general-purpose register E)
- `gf` (general-purpose register F)

- `gg` (general-purpose register G)
- `gh` (general-purpose register H)

One word in Arch-252 is 16 bits wide. Each register is one word wide.

As Arch-252 is a *von Neumann* architecture, it has a **single address space** for both instructions and data that is *word-addressable* (i.e., has two bytes of data per memory address), is little endian, and contains 2^{16} addresses.

Assembly instructions

All Arch-252 assembly instructions are listed below:

Mnemonic	Opcode	Description
<code>jmp op1</code>	<code>0x00</code>	<code>REG[sp] -= 1</code> <code>MEM[REG[sp]] = NextPC</code> <code>PC = op1</code>
<code>set op1, op2</code>	<code>0x01</code>	<code>op1 = op2</code>
<code>add op1, op2</code>	<code>0x02</code>	<code>op1 = op1 + op2</code> <code>REG[tm] = 0x0001 if overflow else 0x0</code>
<code>sub op1, op2</code>	<code>0x03</code>	<code>op1 = op1 - op2</code> <code>REG[tm] = 0xffff if underflow else 0x0</code>
<code>mulu op1, op2</code>	<code>0x04</code>	<code>op1 = op1 * op2 (unsigned)</code> <code>REG[tm] = ((op1 * op2) >> 16) & 0xffff (unsigned)</code>
<code>muls op1, op2</code>	<code>0x05</code>	<code>op1 = op1 * op2 (signed)</code> <code>REG[tm] = ((op1 * op2) >> 16) & 0xffff (signed)</code>
<code>divu op1, op2</code>	<code>0x06</code>	<code>op1 = op1 / op2 (unsigned integer division)</code> <code>REG[tm] = ((op2 << 16) / op2) & 0xffff if op2 != 0</code> <code>else 0x0 (unsigned integer division)</code>

Mnemonic	Opcode	Description
divs op1, op2	0x07	op1 = op1 / op2 (signed integer division) REG[tm] = ((op2 << 16) / op2) & 0xffff if op2 != 0 else 0x0 (signed integer division; rounds toward zero)
modu op1, op2	0x08	op1 = (op1 % op2) if op2 != 0 else 0 (unsigned modulo)
mods op1, op2	0x09	op1 = (op1 % op2) if op2 != 0 else 0 (signed modulo)
and op1, op2	0x0a	op1 = op1 & op2
or op1, op2	0x0b	op1 = op1 op2
xor op1, op2	0x0c	op1 = op1 ^ op2
sr1 op1, op2	0x0d	op1 = op1 >>> op2 (logical right shift) REG[tm] = ((op1 << 16) >> op2) & 0xffff
sra op1, op2	0x0e	op1 = op1 >> op2 (arithmetic right shift) REG[tm] = ((op1 << 16) >> op2) & 0xffff
sll op1, op2	0x0f	op1 = op1 << op2 (logical left shift) REG[tm] = ((op1 << op2) >> 16) & 0xffff
ifany op1, op2	0x10	PC = NextPC if (op1 & op2 != 0) else PC = NextNextPC
ifnon op1, op2	0x11	PC = NextPC if (op1 & op2 == 0) else PC = NextNextPC
ifeq op1, op2	0x12	PC = NextPC if (op1 == op2) else PC = NextNextPC

Mnemonic	Opcode	Description
ifne op1, op2	0x13	PC = NextPC if (op1 != op2) else PC = NextNextPC
ifgtu op1, op2	0x14	PC = NextPC if (op1 > op2) else PC = NextNextPC (unsigned)
ifgts op1, op2	0x15	PC = NextPC if (op1 > op2) else PC = NextNextPC (signed)
ifltu op1, op2	0x16	PC = NextPC if (op1 < op2) else PC = NextNextPC (unsigned)
iflts op1, op2	0x17	PC = NextPC if (op1 < op2) else PC = NextNextPC (signed)
addt op1, op2	0x18	op1 = op1 + op2 + REG[tm] REG[tm] = 0x0001 if overflow else 0x0
subt op1, op2	0x19	op1 = op1 - op2 + REG[tm] REG[tm] = 0xffff if underflow else 0x0
seti op1, op2	0x1a	op1 = op2 REG[gg] += 1 REG[gh] += 1
setd op1, op2	0x1b	op1 = op2 REG[gg] -= 1 REG[gh] -= 1

Note that `NextPC == PC + InstrWidth(MEM[PC])` and that `NextNextPC == NextPC + InstrWidth(MEM[NextPC])`.

op1 (operand 1) and op2 (operand 2) may assume the following forms:

Form	Interpretation
x where x is <code>pc</code> , <code>sp</code> , <code>tm</code> , <code>ga</code> , <code>gb</code> , <code>gc</code> , <code>gd</code> , <code>ge</code> , <code>gf</code> , <code>gg</code> , or <code>gh</code>	<code>REG[x]</code>
$@x$ where x is <code>ga</code> , <code>gb</code> , <code>gc</code> , <code>gd</code> , <code>ge</code> , <code>gf</code> , <code>gg</code> , or <code>gh</code>	<code>MEM[REG[x]]</code>
$@x+k$ where x is <code>ga</code> , <code>gb</code> , <code>gc</code> , <code>gd</code> , <code>ge</code> , <code>gf</code> , <code>gg</code> , or <code>gh</code> and k is either a signed integer that can be expressed in 16-bit two's complement, or <code>0xV</code> where V is sequence of 1-4 hexadecimal symbols (<i>mixed case</i>)	<code>MEM[REG[x] + k]</code>
<code>push</code> (valid only for <code>op1</code>)	<code>MEM[--REG[sp]]</code> (decrement before accessing memory)
<code>pop</code> (valid only for <code>op2</code>)	<code>MEM[REG[sp]++]</code> (increment after accessing memory)
<code>peek</code>	<code>MEM[REG[sp]]</code>
$@peek+k$ where k is either a signed integer that can be expressed in 16-bit two's complement, or <code>0xV</code> where V is sequence of 1-4 hexadecimal symbols (<i>mixed case</i>)	<code>MEM[REG[sp] + k]</code>
<code>[−32768, 65535]</code>	Literal value
<code>0x8000, ..., 0x0, 0x00, 0x000, 0x0000, 0x1, 0x01, ..., 0x3fff</code>	Literal value
$@ [−32768, 65535]$	<code>MEM[literal value]</code>
<code>@0x8000, ..., @0x0, @0x00, @0x000, @0x0000, @0x1, @0x01, ..., @0x3fff</code>	<code>MEM[literal value]</code>

● Machine instruction encoding

Each Arch-252 machine instruction falls into one of four instruction types:

- Type 1 (regular)
- Type 2 (single immediate)
- Type 3 (double immediate)
- Type 4 (special)

All values are represented using two's complement.

Type 1 (regular) instructions are two bytes wide and have the following bit mappings:

- Bits 15-10: Operand 1
- Bits 9-5: Operand 2
- Bits 4-0: Opcode

Type 2 (single immediate) instructions are four bytes wide and have the following bit mappings:

- Bits 23-16: Immediate value
- Bits 15-10: Operand 1
- Bits 9-5: Operand 2
- Bits 4-0: Opcode

Type 3 (double immediate) instructions are six bytes wide and have the following bit mappings:

- Bits 31-24: Immediate value for operand 1
- Bits 23-16: Immediate value for operand 2
- Bits 15-10: Operand 1
- Bits 9-5: Operand 2
- Bits 4-0: Opcode

Type 4 (special) instructions are two bytes wide and have the following bit mappings:

- Bits 15-10: Operand 1
- Bits 9-5: Function
- Bits 4-0: Opcode (*always 00000*)

op1 (operand 1) and op2 (operand 2) values are encoded as follows:

Value	Encoding
ga	0x00
gb	0x01
gc	0x02
gd	0x03
ge	0x04
gf	0x05
gg	0x06
gh	0x07
@ga	0x08
@gb	0x09
@gc	0x0a
@gd	0x0b
@ge	0x0c
@gf	0x0d
@gg	0x0e
@gh	0x0f
@ga+k	0x10 (see below; k is 16-bit immediate)

Value	Encoding
@gb+k	0x11 (see below; k is 16-bit immediate)
@gc+k	0x12 (see below; k is 16-bit immediate)
@gd+k	0x13 (see below; k is 16-bit immediate)
@ge+k	0x14 (see below; k is 16-bit immediate)
@gf+k	0x15 (see below; k is 16-bit immediate)
@gg+k	0x16 (see below; k is 16-bit immediate)
@gh+k	0x17 (see below; k is 16-bit immediate)
push (for op1 only) pop (for op2 only)	0x18
peek	0x19
@peek+k	0x1a (see below; k is 16-bit immediate)
sp	0x1b
pc	0x1c
tm	0x1d

Value	Encoding
<code>k</code>	<code>0x1e</code> (see below; <code>k</code> is 16-bit immediate)
<code>0xKKKK</code>	<code>0x1e</code> (see below; <code>0xKKKK</code> is 16-bit immediate)
<code>@0xKKKK</code>	<code>0x1f</code> (see below; <code>0xKKKK</code> is 16-bit immediate)

● I/O devices

● Memory-mapped I/O

The memory address space is mapped as follows:

- `0x0000-0x8fff` : RAM (instruction and data memory)
- `0x9000-0x9800` : 32x32 RGB display
- `0x9801` : Input button
- `0x9802` : Tick counter
- `0x9803` : RNG source
- `0x9804-0x9fff` : Unused
- `0xa000-0xffff` : RAM (stack memory)

● 32x32 RGB display

A single 32x32 RGB display is available for Arch-252 programs to display graphics with.

Each pixel is represented as a 32-bit value with format `0x00RRGGBB` where `RR`, `GG`, and `BB` are values from 0-255 representing the intensity of red, green, and blue components of the RGB pixel, respectively. Bits 31-24 of the 32-bit value are unused.

The display has its own video RAM; it stores the data of all 1024 pixels with a default of `0x00000000` (*black*). Two sets of 1024 pixels exist (*each called a buffer*) for which one buffer is active at any instant. The currently active buffer determines the pixels shown by the RGB display.

Any pixel update operation performed by the processor will be done on the *inactive* buffer. The processor may issue a *buffer swap* operation to make the currently active buffer inactive and the

other buffer active.

Read operations directed to this device are ignored. There is no way for the processor to select which buffer to make active (*i.e., only a toggle-based selection mechanism is available*).

Input button

A single input button is available for Arch-252 programs to take user input with.

While the button is pressed, its internal 24-bit state is set to `0x000001`. While the button is not pressed, its state is set to `0x000000`. The processor may issue a read operation to get the current state of the button.

Write operations directed to this device are ignored.

Tick counter

A 16-bit tick counter is available for Arch-252 programs that can be read from by the processor to get the total number of clock cycles that has passed (*modulo 16*).

This device is intended to be used for busy-waiting in Part 3 to ensure that each game loop starts at roughly the same delay from the previous one.

Write operations directed to this device are ignored.

RNG source

A device that provides a random 24-bit value upon being read from is available for Arch-252 programs as a convenient source of randomness.

Write operations directed to this device are ignored.

Tasks

As mentioned earlier, each member is expected to be the *main contributor* for at least one task below.

Part 1: RISC-V-Based Assembler

Overview

Create a RISC-V-based assembler that converts Arch-252 assembly code into a binary file containing equivalent Arch-252 machine instructions.

Your entire code must be in a single file named `assembler.s` inside the `part1` directory.

Execution

Your code is expected to work with [v1.01 of the offline CS 21 25.2 Ripes fork](#) and should be executed via the command line as follows (replacing `ripes` with the path to your locally installed Ripes fork):

```
ripes --src assembler.s --mode cli -t asm --proc RV32_5S --isaexts M
```

Upon execution, syscall code 63 (`read syscall`) must be invoked with file descriptor equal to `0` to take a string input from the user via `stdin` that is at most 300 characters. This string will be referred to as `<input-filepath>`.

After taking `<input-filepath>`, the `read` syscall must again be invoked in the same manner to take another string input from the user that is also at most 300 characters. This string will be referred to as `<output-filepath>`.

`<input-filepath>` filepath is taken to be the absolute or relative path to a file (*ideally with the extension `.252asm`*) containing an Arch-252 assembly program to assemble. `<output-filepath>` is the expected path to the output file (*ideally with the extension `.252bin`*).



Tip: File I/O with Ripes

Syscall code 1024 (`open syscall`) can then be used to open the said files for reading and writing, respectively.

Reading can be done with the `read` syscall using the file descriptor given by the corresponding `open` syscall. Writing can be done similarly via syscall code 64 (`write syscall`).



Tip: Streamlined testing

To streamline testing, an input file (say `input.txt`) can be created where the first line is `<input-filepath>` and the second line is `<output-filepath>`.

For example, `<input-filepath>` being `input.252asm` and `<output-filepath>` being `output.252bin` would result in the following `input.txt` (replacing `ripes` with the path to your locally installed Ripes fork):

```
input.252asm
output.252bin
```

This can then be fed to Ripes as user input via terminal file redirection:

```
ripes --src assembler.s --mode cli -t asm --proc RV32_5S --isaexts M < input.txt
```



`.raw` directive

`.raw` is an Arch-252 assembler directive that allows assembly programs to statically specify raw words in fixed memory addresses. The `.raw` directive is written as follows:

```
.raw <word>
```

where `<word>` is either `0x` followed by 1-4 hexadecimal symbols, or an integer in the range `[−32,768, 65,535]`.

`.raw` is an *emitting directive* (i.e., it results in data being written to the emitted binary file).



`.addr` directive

`.addr` is an Arch-252 assembler directive that allows assembly programs to set the current memory address to be written to. The `.addr` directive is written as follows:

```
.addr <a>
```

where `<a>` is either `0x` followed by 1-4 hexadecimal symbols, or an integer in the range `[0, 65,535]`. `<a>` is taken to be the next address to be written to by the following instruction or emitting directive.

If `.addr` sets the current address to an address that has already been written to previously, the next instruction or emitting directive will simply overwrite the existing data.

`.addr` is a *nonemitting directive* (i.e., it does **not** result in data being written to the emitted binary file).

● Assembly file format

Each line in a `.252asm` file is expected to be one of the following:

- Instruction
- Directive
- Label
- Comment

Simplifying assumptions

To simplify parsing, each line is expected to have only one of the above, or be an empty line consisting of zero or more spaces.

Lines longer than 100 characters are **not** expected to be handled by the assembler (i.e., no expected behavior).

All assembly files to be used during testing will be at most 20,000 lines long.

Instruction lines start with zero or more spaces, followed by exactly one of the assembly instructions stated above, and zero or more spaces until the end of the line.

Directive lines start with zero or more spaces, followed by exactly one of the directives stated above, and zero or more spaces until the end of the line.

Label lines start with zero or more spaces, followed by **1-50 character** that are neither colons nor spaces (*where the string is referred to as the **label***), then exactly one colon (*optionally zero or more spaces before the end of the line*). During checking, no label will be equal to an existing instruction or directive.

Comment lines start with zero or more spaces, followed by a `#`, and followed by any number of characters (*limited by the 100-character line limit*) until the end of the line. Comments are **not** converted into any artifact in the machine code emitted by the assembler.

● Binary file format

Each instruction or emitting directive is converted into a group of bytes that is placed in the binary file in order of appearance. In particular, the i^{th} group of bytes that appears in the binary file should correspond to the i^{th} instruction or emitting directive.

The bytes in each group appear in little endian order in the binary file.

The emitted binary file must **always** be $2^{16} = 65,536$ bytes wide. Unspecified bytes are assumed to be equal to `0x00`.

Error reporting

The assembler must terminate without emitting a binary file upon processing the first occurrence of any of the following errors:

- **Invalid instruction** (*the line does not contain a valid instruction or directive*)
- **Invalid argument** (*the line contains a valid instruction or directive, but has at least one argument that is invalid*)

Simplifying assumptions

You may assume that `<input-filepath>` and `<output-filepath>` will always be valid (*i.e., no error handling is expected for invalid paths*).

The following text must be output to `stderr` (*write syscall with file descriptor 2*) if either an *invalid instruction* or *invalid argument* error is detected:

```
Error in line <n>: <reason>
```

where `<n>` is the line closest to the top of the file that has an error and `<reason>` is either `Invalid instruction` or `Invalid argument`.

If an error occurs, the exit code of the program must be set to `1` for `Invalid command-line arguments`, `2` for `Invalid instruction` or `3` for `Invalid argument`. If no error occurs, the exit code must be set to `0`.

Tip: Syscall code 93

The `exit2` syscall (*syscall code 93*) is similar to the `exit` syscall, but allows specifying the exit code via `a0`.

Part 2: Pyxel-Based Arch-252 Emulator

Overview

Create a Python-based program that uses [Pyxel](#) to implement an *emulator* for Arch-252. An emulator

The emulator is supposed to be able to load a *binary file* in the exact format emitted by the assembler in Part 1. It must use Pyxel to emulate the 32x32 RGB display and the input button expected for Arch-252 as defined previously.

Your entire code must be in a single file named `emulator.py` inside the `part2` directory.

The Pyxel FPS value must be hardcoded at `60`.

Tip: Breaking the dependency chain

You are strongly encouraged to hand-write `.252bin` files with the help of simple Python scripts instead of waiting for the Part 1 deliverable to be fully functional.

Once both Part 1 and Part 2 deliverables are working, you may use the output of Part 1 as the input to Part 2 for correctness verification of both implementations.

Execution

Your program must be executable using the following command (*replacing `python3` with the path to the Python interpreter of your system*):

```
python3 emulator.py <bin-filepath> <ipf> <seed>
```

where `<bin-filepath>` refers to the filepath to the binary file containing the Arch-252 machine instructions of the program to emulate, `<ipf>` is the number of Arch-252 instructions to perform during in a single Pyxel frame, and `<seed>` pertains to the integer randomization seed used by the emulated RNG device (*details below*).

32x32 RGB display

The Pyxel window should have a grid with 32 rows and 32 columns that represents the RGB display.

Since Pyxel supports up only up to 16 colors at any instant, the 24-bit RGB color scheme expected by Arch-252 programs has to be approximated using a 16-color palette.

A good approximation for this is the [DawnBringer 16 palette](#). Palette details are given below for your convenience ([source](#)):

Hex code (#rrggbb)	Color	Sample
#4e4a4e	emperor	
#442434	livid brown	
#30346d	rhino	
#854c30	mule fawn	
#346524	woodland	
#757161	pablo	
#d04648	flush mahogany	
#597dce	danube	
#d27d2c	brandy punch	
#6daa2c	olive drab	
#8595a1	regent gray	
#d2aa99	eunry	
#6dc2ca	downy	
#dad45e	tacha	

Hex code (#rrggbb)	Color	Sample
#140c1c	ebony	
#deeed6	zanah	

To quantize the 24-bit RGB value into a color in the DawnBringer 16 palette, you are to use the following color quantization scheme:

$$\text{quantize}(c) = \arg \min_{p \in P} [(r_c - r_p)^2 + (g_c - g_p)^2 + (b_c - b_p)^2]$$

where P is the set containing all 16 RGB colors of the DawnBringer 16 palette, c is the RGB color to quantize, and r_k , g_k , and b_k are the red, green, and blue components of RGB color k , respectively.

You are free to set the size of the RGB display relative to the Pyxel window.

The MMIO mapping for the RGB display stated earlier must be followed. The buffer flipping functionality described earlier must also be implemented properly.

● Input button

Emulation of the input button is done by simply mapping the `Spacebar` keyboard key to the input button. As long as the `Spacebar` key is pressed, the input button should also be pressed.

The MMIO mapping for the input button stated earlier must be followed.

● Tick counter

Emulation of the tick counter should be done by incrementing a counter *after* executing a single instruction (*i.e.*, reading from the tick counter during the very first tick should give a value of 0).

It is expected for the 16-bit counter to overflow to zero after reaching 65,535.

The MMIO mapping for the tick counter stated earlier must be followed.

● RNG source

Emulation of the RNG source should be done by instantiating a singleton `Random` object in the built-in `random` package of Python, and initializing it with the `<seed>` value given in the command-line arguments.

The MMIO mapping for the RNG source stated earlier must be followed.

Information panel

The area to the right of the RGB display should contain information about the running program. In particular, the following should be shown:

- Value of `<ipf>`
- Value of `<seed>`
- Number of executed instructions so far
- Current value of tick counter
- Current value of `pc` in hex
- Current *assembly instruction* under `pc` with proper syntax
- Current value of `sp` in hex
- Current value of `tm` in hex
- Current value of `ga` in hex
- Current value of `gb` in hex
- Current value of `gc` in hex
- Current value of `gd` in hex
- Current value of `ge` in hex
- Current value of `gf` in hex
- Current value of `gg` in hex
- Current value of `gh` in hex

During the end of the first tick, `<ipf>` instructions should have been executed and the resulting state should be displayed.

Part 3: Simplified Flappy Bird in Arch-252 Assembly

Overview

Create a simplified [Flappy Bird](#) program in Arch-252 assembly.

Your entire code must be in a single file named `flappybird.252asm` inside the `part3` directory.

● Game loop

The game must operate using a game loop for which the start of each iteration takes roughly the **same number of ticks**.

Each iteration of the game loop should do the following:

1. Check for user input
2. Update game state
3. Update game state
4. Perform a buffer swap of the RGB display
5. Busy-wait until enough cycles have passed to start next iteration (*i.e., waste cycles in a loop until time of next iteration*)

While you are free to set the number of ticks between game loop iterations, you must ensure that it is roughly followed. It follows that the said number of ticks must be *at least as long* as the longest time needed for a single iteration to do the operations stated above.

● Player-controlled bird

Zero-indexed rows and columns

In this document, rows and columns will be referred to using zero-based indexing.

The player-controlled bird is a 2x2 square with its top-left cell initially positioned at row 15, column 0.

The color of the bird should be reasonably distinguishable from the colors of all other entities on the grid. Additionally, it must also be reasonably distinguishable from any of the colors of the *DawnBringer 16* color palette described in [32x32 RGB display section of Part 2](#).

If the input button has *just* been pressed during a game loop iteration, it must set the vertical velocity of the bird to a fixed value that should result in the bird going upward (*i.e., a negative horizontal velocity*). While you are free to set the actual value of the vertical velocity, the resulting game difficulty must be reasonable.

There must be a periodic addition of a constant value to the vertical velocity of the bird (*i.e., constant acceleration due to gravity ideally applied during each game loop iteration*). While you are free to set the acceleration value, the resulting game difficulty must be reasonable.

 Tip: Simulating fractional numbers using integers

You may employ a **fixed-point representation** of a subset of numbers with fractional parts using integers.

For example, a fixed-point representation that allows for two digits after the decimal point may represent the fractional value 25.2 as the integer 2500 with arithmetic being possible via rounding adjustments before and after computation.

Holding the input button without releasing across game loop iterations must have no effect on the vertical velocity of the bird.

 Pipes

The goal of the game is to maneuver the bird for as long as possible through vertical openings between *pairs of pipes* that are randomly generated (*moving from right to left to facilitate the illusion of movement*) without the bird hitting any of the pipes or the ground.

The game should start with three sets of *pipe pairs*, each with exactly eight cells separating them from the next pair with the leftmost pair starting at column 10.

Each pipe must have a width of two cells and a height of *at least* one cell. The vertical opening that separates pairs of pipes from each other must **always** be 12 cells tall and starts at row h that is randomly generated per pipe pair.

In particular, the 32-cell height containing a pipe pair is divided into three parts:

1. Upper pipe with height h
2. 12-cell vertical opening
3. Lower pipe with height equal to remaining vertical cells

h must be a randomly generated value in the range $[1, 19]$ that is derived in some way from the 16-bit value read from the RNG source.

Pipes must move in a constant speed from right to left (*i.e., they must have a negative horizontal velocity*). While you are free to set the actual value of the horizontal velocity (*which can be "fractional" as explained above*), the resulting game difficulty must be reasonable.

The color of the pipes should be reasonably distinguishable from the colors of all other entities on the grid. Additionally, it must also be reasonably distinguishable from any of the colors of the *DawnBringer 16* color palette described in [32x32 RGB display section of Part 2](#).

● Background

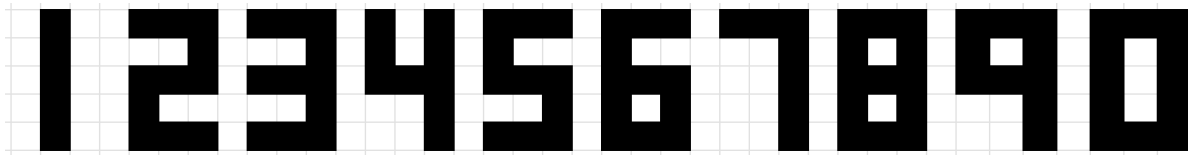
The color of the background should be reasonably distinguishable from the colors of all other entities on the grid. Additionally, it must also be reasonably distinguishable from any of the colors of the *DawnBringer 16* color palette described in [32x32 RGB display section of Part 2](#).

● Scoring

Whenever the left half of the bird reaches the right half of a pipe pair, the score of the player must be incremented by 1.

The current score of the player must be shown in base 10 at the top-right corner of the display, initially with a value of `00`. The displayed score is fixed to be only two digits wide. Only the last two digits of scores with 3+ digit should be displayed.

Each digit is 3x5 and should appear as follows:



The first digit of the two-digit score must have its top-left cell at row 1, column 24 while the top-left cell of the second at row 1, column 28.

The color of the text should be reasonably distinguishable from the colors of all other entities on the grid. You are allowed to use one of the colors of the *DawnBringer 16* color palette.

● Game flow

The game must start with the bird and the three pipe pairs at their initial positions. None of the entities will start moving (*i.e., velocity and acceleration will not yet apply*) until the input button is pressed.

When the bird is in contact with any pipe or the ground (*where contact means overlapping cells in the case of pipes and having the top half of the bird be at row 31 for the ground*), movement should stop (*i.e., velocity and acceleration will stop being applied*), signifying that the game is over.

While the game is over, pressing the input button should result in a new game state with the score reset to `00`, the bird back in its initial position, and a new set of three starting pipe pairs randomly generated and at their starting positions.

● Part 4: Digital-Based Arch-252 Processor

Live Feature Addition Recording

In addition to the deliverables per task above, each **main contributor** is expected to submit a live recording of a *feature addition* to their assigned deliverable.

Recording rules

- Live face of presenter must be shown throughout recording together with screen recording; Zoom may be used to easily set this up
- For code-based deliverables, the code shown in the video must be reasonably legible
- The video must be taken in one go (*i.e., no cuts/pauses*)
- Explanations must be given as to why each addition or edit done is necessary
- Explanations must be the own words of the presenter (*i.e., no reading from a script*)
- While rehearsing the implementation of the new feature beforehand is allowed for code-based deliverables, the code must **not** be copied from some reference during the live recording
- Related portions of the deliverable that are from AI output must be mentioned explicitly, complete with reference to the exact part of the LLM log (see [LLM Policy section](#)) showing the AI output
- Video must include demo at the end proving that the implementation works as intended
- Each video must take at most 30 minutes and must be uploaded separately (should **not** be spliced into a single compilation)

Failed rule adherence

All members of a group for which *at least one recording* that fails to adhere to the rules above **will be given a grade of INC**.

The **INC** grade will be resolved only through a **synchronous in-person meeting** to be scheduled with the CS 21 25.2 handlers during the midyear for the main contributors of the offending recordings.

Part 1 addition: `.def` directive

Implement the `.def` directive which is a nonemitting Arch-252 assembler directive that allows assembly programs to statically specify reusable aliases for recurring values. The `.def` directive

is written as follows:

```
.def <name> <value>
```

where `<name>` and `<value>` are strings without spaces or newlines that are each **at most 40 characters wide**.

Simplifying assumptions

Each assembly program may have at most 10 `.def` lines.

You may assume that all instances of `.def` in the assembly source are given before any other line and that each `<name>` will not conflict with any existing instruction or directive name.

Your implementation does **not** have to be efficient (*e.g., linear search on a static array is acceptable*).

While the error handling mechanism must be modified to ensure that it does **not** interfere with the addition of aliases (*you may do so by disabling the invalid argument check for the demo*), there is no need to handle inexistent aliases.

Part 2 addition: `sp` region details

Extend the information panel such that the following details are displayed:

- Word under address `sp - 3` in hex
- Word under address `sp - 2` in hex
- Word under address `sp - 1` in hex
- Word under address `sp` in hex
- Word under address `sp + 1` in hex
- Word under address `sp + 2` in hex
- Word under address `sp + 3` in hex

If any address above goes out-of-bounds of the address space, the value displayed for that address should be blank.

Existing text in the information panel should be moved to make ample space for the new details.

Part 3 addition: Y-coordinate display

Show the Y-coordinate of the bird (*i.e., which zero-indexed row it currently is on*) as a two-digit number similar to the score.

The top-left cell of the first digit must start at row 26, column 24 while the top-left cell of the second must start at row 26, column 28.

The value shown must be the Y-coordinate of the bird at the end of a game loop iteration.

The color of the text should be reasonably distinguishable from the colors of all other entities on the grid. You are allowed to use one of the colors of the *DawnBringer 16* color palette.

Part 4 addition: Information panel

The main circuit should have a region containing circuit components showing the following:

- Current value of 16-bit tick counter using four seven-segment hex displays
- Current instruction (*e.g., ifgtu*) via LEDs (*one per instruction for all 27 instructions*)

LLM Policy

While you are allowed to use LLMs to assist you, you are **required** to include a text or PDF file for each part containing **all** prompts and responses used for the project (*see the next section for the required filenames*).

If LLMs were **not** used for a specific part (*e.g., Part 1*), the said text or PDF file should **not** be present for the said part.

As mentioned, you are **required** to state in the live demo recording whether each relevant portion is your own work or from an LLM.

For cases in which the presenter of a live demo recording clearly does **not** understand the implementation in question, LLM logs must clearly show that the relevant portion of the implementation is sourced from an LLM.

If the LLM logs do **not** show that the portion of the implementation in question is from LLM use, then it will be assumed that the implementation is from a person who is **not** the main contributor (*i.e., academic dishonesty*) and will be dealt with accordingly.

Submission

Github Classroom repository

For each group, assign one person to create a Github Classroom repository for the group here (any unique group name will do): <https://classroom.github.com/a/bkugNLXB>

After the repository has been made, all other group members should click the link above and select the correct group.

The Github Classroom repository is meant to be a *monorepo* that contains the following at its root:

- README.md
- llm-part1.pdf or llm-part1.txt
- llm-part2.pdf or llm-part2.txt
- llm-part3.pdf or llm-part3.txt
- llm-part4.pdf or llm-part4.txt
- part1/
- part2/
- part3/
- part4/

README.md must contain the following:

1. The names of your group members
2. Which deliverable each group member is the *main contributor* to
3. AI usage attribution as previously explained
4. Links to video documentation as previously explained
5. (optional) An overview of the project for portfolio purposes



Required repository visibility

The visibility of the repository must be set to **private**.

You are required to first seek permission from the CS 21 25.2 handlers to make this public for portfolio purposes.



Peer evaluation

You are to evaluate each group member based on three separate aspects on or before **June 8 (M), 11:59 PM**:

- Reliability (30%)
- Collaboration and Communication (30%)
- Live recording (40%)

A Google Form will be made available with more information regarding each aspect.

● Grading scheme

Your grade for the project will be computed as follows:

$$\bullet \text{ } TotalScore = (0.25 \times IndivScore) + (0.75 \times GroupScore \times PeerEval)$$

where

- k refers to the single part for which you are the main contributor
- $IndivScore = IndivScore_k = (0.5 \times TestCases_k) + (0.5 \times VideoScore_k)$
 - $TestCases_k$ is the score of Part k against test cases; range of $[0, 100]$
 - $VideoScore_k$ is the score of the live recording for your part; range of $[0, 100]$
- $GroupScore = \sum_{i=1}^4 (TestCases_i \times 25)$
- $PeerEval$ is the average percentage of all submitted peer evaluations for you after dropping the lowest score; range of $[0, 1]$
- $TestCases_k$ effectively appears twice in $TotalScore$

For students with **two parts** k and m which they are the main contributor to:

$$\bullet \text{ } IndivScore = (0.5 \times IndivScore_k) + (0.5 \times IndivScore_m)$$

● Deadline

Submit by **June 7 (Su), 11:59 PM.**

THE PART 2 PLAN

Phase 0: Read the ISA and Produce Internal Documentation

Before writing code, extract from the spec:

Registers

```
pc
sp
tm
ga
gb
gc
gd
ge
gf
gg
gh
```

11 registers total.

Create a Python mapping:

```
REG_PC = 0
REG_SP = 1
...
REG_GH = 10
```

or

```
regs = {
    "pc": 0,
    ...
}
```

MMIO Map

0x0000 - 0x8fff	RAM
0x9000 - 0x9800	Display
0x9801	Button
0x9802	Tick Counter
0x9803	RNG
0xa000 - 0xffff	Stack RAM

Create constants immediately.

Instruction Table

Create one master table:

```
INSTRUCTIONS = {  
    0x00: "jmp",  
    0x01: "set",  
    ...  
}
```

This will later drive:

- decoding
 - disassembly display
 - execution
-

Phase 1: Binary Loader

Goal:

```
Read .252bin  
Load memory  
Exit
```

No emulator yet.

Memory:

```
memory = [0] * 65536
```

because address space is 2^{16} words.

Read binary:

```
with open(path, "rb") as f:
```

Load little-endian words.

Verify:

```
memory[0]  
memory[1]  
...
```

with test files.

Phase 2: Register File and CPU Skeleton

Implement:

```
class CPU:
```

with

```
regs  
memory  
tick_counter  
rng
```

Initialize:

```
pc = 0  
sp = 0xffff
```

(or whatever the spec/testing assumes)

Create:

```
step()
```

which will eventually execute one instruction.

At this point:

```
cpu.step()
```

does nothing.

Phase 3: Instruction Fetch

Goal:

Fetch opcode correctly.

Read first word:

```
instr_word = memory[pc]
```

Extract:

```
opcode = instr_word & 0x1f
op2     = (instr_word >> 5) & 0x1f
op1     = (instr_word >> 10) & 0x3f
```

Create:

```
decode()
```

returning

```
DecodedInstruction(...)
```

Phase 4: Instruction Width Detection

Critical phase.

Arch-252 has:

```
Type 1 = 2 bytes
Type 2 = 4 bytes
Type 3 = 6 bytes
Type 4 = 2 bytes
```

You need:

```
instruction_width()
```

because:

```
NextPC
NextNextPC
```

depend on it.

Many branches will break if this is wrong.

Phase 5: Operand Resolution

This is the hardest architectural component.

Create:

```
read_operand(opcode)
write_operand(opcode, value)
```

Support:

```
register
@register
@register+k
push
pop
peek
```

```
@peek+k  
literal  
@literal
```

Everything else should call these helpers.

Goal:

Instruction handlers should look like:

```
a = read(op1)  
b = read(op2)  
  
write(op1, a + b)
```

not:

```
if op1 == ...  
if op2 == ...
```

everywhere.

Phase 6: Arithmetic Instructions

Implement first:

```
set  
add  
sub  
and  
or  
xor
```

These are easiest.

Verify:

```
register-register  
register-memory  
memory-register
```


cases.

Phase 7: Remaining ALU Instructions

Add:

```
mulu  
muls  
divu  
divs  
modu  
mods  
sll  
srl  
sra
```

Pay special attention to:

```
tm
```

overflow behavior.

This phase will likely consume most debugging time.

Phase 8: Branches and Control Flow

Implement:

```
jmp  
ifeq  
ifne  
ifgtu  
ifgts  
ifltu  
iflts  
ifany  
ifnon
```

Build a helper:

```
next_pc()  
next_next_pc()
```

because every branch uses them.

Phase 9: seti / setd

Implement separately.

These have special behavior:

```
gg++  
gh++
```

or

```
gg--  
gh--
```

Keep isolated from ordinary instructions.

Phase 10: MMIO Layer

Create:

```
memory_read(addr)  
memory_write(addr)
```

Never access memory directly again.

Inside:

```
if addr == BUTTON:
```

etc.

Implement:

Button

```
0 or 1
```

Tick Counter

Returns:

```
tick & 0xffff
```

RNG

Use:

```
random.Random(seed)
```

Phase 11: Headless Emulator

Before touching Pyxel:

Create:

```
python emulator.py file.bin 1 123
```

and run entirely in terminal.

No graphics.

No GUI.

Just:

```
for i in range(1000):  
    cpu.step()
```

This phase should get all instruction semantics correct.

Phase 12: Display Buffers

Implement:

```
front_buffer  
back_buffer
```

1024 pixels each.

Implement:

```
swap_buffers()
```

exactly as specified.

Test separately.

No Pyxel yet.

Phase 13: Pyxel Window

Now finally:

```
pyxel.init(...)
```

Render:

```
32x32 grid
```

Initially:

```
draw random colors
```

to verify rendering.

Phase 14: Color Quantization

Implement nearest-color search over the DawnBringer palette.

Create:

```
quantize(rgb)
```

Test separately.

Phase 15: Connect MMIO Display to Pyxel

Memory writes:

```
0x9000-0x9800
```

must update inactive buffer.

Render active buffer.

At this point:

Flappy Bird can eventually display graphics.

Phase 16: Information Panel

Implement:

```
ipf
seed
instructions executed
tick counter
pc
current instruction
all registers
```

This is mostly UI work.

Leave room for the recording feature.

Phase 17: Video Feature (Do Last)

Add:

```
sp-3
sp-2
sp-1
sp
sp+1
sp+2
sp+3
```

display.

Do not implement this until recording day.

It's almost completely independent of the rest of the emulator.

Recommended Timeline

Week 1

- ISA tables
- memory
- loader
- fetch/decode

Week 2

- operand system
- arithmetic instructions
- branches

Week 3

- MMIO
- display buffers
- headless testing

Week 4

- Pyxel frontend
- information panel

- polish

Final Week

- recording feature
- bug fixes
- integration with Part 3

This order minimizes risk because the CPU semantics are verified before any GUI code gets involved. Most emulator projects become difficult when execution bugs and rendering bugs are introduced simultaneously.